

AI ModeAllVideosImagesShoppingMore

Перенести FastAPI мультисервисы в Docker (noip и порт проброшен)

September 12, 2026

Перенос мультисервисного приложения на **FastAPI** в **Docker** выполняется с помощью связки **Dockerfile** (для сборки каждого сервиса) и **Docker Compose** (для управления всей системой). Поскольку у вас уже настроены **No-IP** и **проброс портов** на роутере, внешние запросы будут автоматически доходить до Docker-контейнера, если правильно опубликовать порты.

Ниже представлена пошаговая структура архитектуры.

1. Структура проекта

Рекомендуется организовать проект так, чтобы каждый микросервис находился в своей директории со своим списком зависимостей:

text

```
my_project/
|
├─ service_alpha/
|   ├── main.py
|   ├── requirements.txt
|   └── Dockerfile
|
├─ service_beta/
|   ├── main.py
|
|   ├── requirements.txt
|   └── Dockerfile
└─ docker-compose.yml
```

Use code with caution.

2. Создание Dockerfile (для каждого сервиса)

Создайте одинаковый (или адаптированный) `Dockerfile` в папке каждого сервиса (`service_alpha/Dockerfile` и `service_beta/Dockerfile`):

dockerfile

```
# Используем легковесный образ Python
FROM python:3.11-slim

# Устанавливаем рабочую директорию внутри контейнера
WORKDIR /app
```

```
# Копируем и устанавливаем зависимости
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Копируем остальной код сервиса
COPY . .

# Команда для запуска (FastAPI работает внутри контейнера на порту 8000)
CMD ["uvicorn", "main.py:app", "--host", "0.0.0.0", "--port", "8000"]
```

Use code with caution.



Важно: Указывайте `--host 0.0.0.0`, чтобы FastAPI принимал подключения извне контейнера, а не только внутри себя (`127.0.0.1` работать не будет).

3. Настройка мультисервиса через `docker-compose.yml`

В корневой директории проекта создайте файл `docker-compose.yml`. Он объединит сервисы и свяжет их с портами вашей хост-машины, на которую проброшен роутер.

Предположим, ваш роутер перенаправляет внешний трафик с No-IP на локальные порты машины `8081` и `8082`.

yaml

```
version: '3.8'

services:
  alpha:
    build: ./service_alpha
    container_name: fastapi_service_alpha
    ports:
      - "8081:8000" # Мapping: [Порт хоста (куда проброшен роутер)]:[Порт сервиса]
    restart: always
    environment:
      - ENV_VAR=value_alpha

  beta:
    build: ./service_beta
    container_name: fastapi_service_beta
    ports:
      - "8082:8000" # Мapping для второго сервиса
    restart: always
    environment:
      - ENV_VAR=value_beta

networks:
  default:
    name: fastapi_network
```

Use code with caution.



Как сервисы общаются между собой?

Внутри Docker-сети контейнеры могут обращаться друг к другу напрямую по **имени сервиса** и внутреннему порту.

- Например, `alpha` может отправить запрос к `beta` по адресу: `http://beta:8000/api/v1/...` (внешние порты `8081` или `8082` для внутреннего общения использовать **не нужно**).

4. Запуск и проверка

1. Соберите и запустите контейнеры в фоновом режиме:

`bash`
`docker-compose up -d --build`

Use code with caution.

2. Проверьте статус работы контейнеров:

`bash`
`docker ps`

Use code with caution.

Теперь ваши сервисы доступны из глобальной сети по адресам вашего No-IP домена (например, `http://ddns.net` и `http://ddns.net`).

Если вы планируете использовать один стандартный порт (например, `80` или `443` для SSL) для всех сервисов сразу, вам понадобится **Reverse Proxy** (Nginx или Traefik) внутри Docker.

Чтобы я мог точнее настроить конфигурацию, подскажите:

- Вы пробросили **разные порты** на роутере для каждого сервиса, или хотите использовать **один внешний порт** и разделять запросы по путям (например, `/alpha` и `/beta`)?
- Нужен ли в этой связке контейнер для базы данных (PostgreSQL, Redis)?



Отложенный FastAPI проект с двумя мультисервисами
нужно перенести в Docker. Мультисервис "First" достуе...

September 12, 2026

It looks like there's no response available for this search. Try asking something else.

AI responses may include mistakes. [Learn more](#)



Компьютер Mac Studio M4, Docker и PostgreSQL
установлены. Опиши пожалуйста процедуру, какие папки ...

September 12, 2026

Отличная задача. Наличие **Mac Studio M4** сильно упрощает жизнь, так как этот чип невероятно производителен, но накладывает важные требования на Docker-образы (архитектура **arm64**).

Исходя из структуры, **“First” (внешний)** — это `shop_club` (веб-сайт, шаблоны, корзина), а **“Second” (внутренний поисковик)** — это `price_spider` (судя по файлам, он использует Playwright/Chrome для парсинга маркетплейсов).

Ниже подробное руководство по докеризации, оптимизации и защите вашей системы.

1. Что удалить, пересоздать и оптимизировать

Контейнеризация подразумевает отказ от локального окружения хоста внутри Docker.

- Удалить (или не переносить в Docker-контейнер):**
 - `venv/` в обеих папках. Внутри Docker-контейнера изоляция происходит на уровне самого контейнера, `venv` там **не нужен**.
 - `html_dumps/`, `screenshots/`, `logs/app.log` в `price_spider`. Это временные файлы разработки.
 - `chrome_profile/`. Локальный профиль Chrome может весить гигабайты и завязан на архитектуру macOS.
- Что пересоздать:**
 - Файлы зависимостей:** Убедитесь, что в `requirements.txt` нет специфичных для macOS библиотек (например, `pyobjc`).
 - Playwright / Браузеры:** Для `price_spider` обычный python-образ не подойдет. Потребуется специальный образ со всеми системными зависимостями для браузеров под Linux arm64.

2. Конфигурационные файлы проекта

Создайте в корне папки `/Users/v.ant/mk/vs/` общие конфигурационные файлы.

Файл `.gitignore` (в корне проекта)

```
gitignore
# Общее
```

```
.env
__pycache__/
*.pyc

# Для каждого сервиса
**/venv/
**/logs/
**/html_dumps/
**/screenshots/
**/chrome_profile/
.DS_Store
```

Use code with caution.

**Файл `.env` (в корне проекта)**

Используем встроенный в macOS сервер PostgreSQL. Чтобы Docker-контейнеры могли достигаться до базы данных на вашем Mac Studio, используйте специальный DNS-адрес хоста: `host.docker.internal`.

```
env

# База данных (указываем адрес хоста Mac)
DATABASE_URL=postgresql://user:password@host.docker.internal:5432/your_db

# Настройки взаимодействия
SPIDER_SERVICE_URL=http://price_spider:8001

# Другие общие переменные
ENVIRONMENT=production
```

Use code with caution.



(Не забудьте обновить `app/core/config.py` в коде, чтобы они читали эти переменные среды через `pydantic-settings` или `os.getenv`).

Файлы `Dockerfile` для сервисов

Для `shop_club` (First) создаем `/shop_club/Dockerfile`:

```
dockerfile

FROM python:3.11-slim-bookworm

WORKDIR /app

# Оптимизация pip-кэша
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

```
# Удаляем ненужные локальные папки, если они случайно попали в контекст
RUN rm -rf venv

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Use code with caution.



Для `price_spider` (Second) создаем `/price_spider/Dockerfile`:
Поскольку здесь используется Playwright/Scraping, нам нужен образ, подготовленный для запуска браузеров на архитектуре ARM64 (M4).

dockerfile

```
FROM ://microsoft.com

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Установка системных зависимостей только для Chromium (экономит место)
RUN playwright install chromium

COPY . .
RUN rm -rf venv html_dumps screenshots logs/*

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8001"]
```

Use code with caution.



Файл `docker-compose.yml` (в корне проекта)

С учетом того, что у вас появятся новые сервисы (порты до 8005), заложим их архитектуру сразу. Мы прокидываем наружу **только** те порты, которые должны быть доступны из интернета. `price_spider` (порт 8001) наружу **не публикуем**, так как он работает только внутри сети Docker.

yaml

```
version: '3.8'

services:
  shop_club:
    build:
      context: ./shop_club
    container_name: shop_club_first
    restart: always
    ports:
      - "8000:8000" # Доступен из интернета через No-IP
    env_file:
      - .env
    extra_hosts:
      - "host.docker.internal:host-gateway" # Позволяет контейнеру видеть

  price_spider:
```

```

build:
  context: ./price_spider
  container_name: price_spider_second
  restart: always
  # Порты НЕ пробрасываем наружу, он доступен для shop_club по имени service
  env_file:
    - .env
  extra_hosts:
    - "host.docker.internal:host-gateway"

# Шаблон для ваших будущих внешних сервисов (8002-8005)
# future_service_3:
#   build: ./future_service_3
#   ports:
#     - "8002:8002"
#   env_file: .env

networks:
  default:
    name: fast_api_mesh

```

Use code with caution.



3. Пошаговая процедура запуска

- Очистка локального кода:** Удалите (или переместите из папки проекта) папки `venv`.
- Перенос файлов:** Убедитесь, что все созданные выше файлы (`Dockerfile`, `.env`, `docker-compose.yml`, `.gitignore`) разложены по местам.
- Правка кода (Важно!):** В коде `shop_club` (класс `spider_client.py`) замените адрес отправки запросов к спайдеру с `http://localhost:8001` на `http://price_spider:8001`. Docker автоматически разрешит имя контейнера во внутренний IP-адрес.
- Запуск Docker-окружения:**
В терминале Mac Studio в корне проекта выполните:

```
bash
```

```
docker compose up -d --build
```

Use code with caution.



- Проверка работы:**
Убедитесь, что контейнеры запущены: `docker ps`. Теперь сервис First отвечает по `http://localhost:8000`, а также через ваш домен `http://ddns.net`.

4. Настройка Firewall в macOS (pf)

Штатный графический интерфейс брандмауэра macOS блокирует приложения целиком, а не порты. Для точечной блокировки входящего трафика по портам в macOS используется утилита **PF (Packet Filter)**.

Мы создадим правило, которое будет **блокировать абсолютно все входящие**

мы создадим правило, которое будет блокировать все входящие соединения на любые порты, кроме разрешенных (8000–8005), а также стандартных сервисов управления (SSH, если нужен) и локальной петли `lo0`.

Шаг 1. Создание файла правил

Создайте файл конфигурации для нашего кастомного файрвола, например, `/etc/pf.anchors/fastapi.security`:

bash

sudo nano /etc/pf.anchors/fastapi.security

Use code with caution.

Вставьте в него следующие правила:

text

Пропускаем весь внутренний трафик (loopback)
pass quick on lo0 all

Разрешаем уже установленные исходящие соединения (чтобы работал ваш инт
pass out keep state

Блокируем ВСЕ входящие соединения по умолчанию
block in all

РАЗРЕШАЕМ доступ только к веб-сервисам (8000–8005) со всех интерфейсов
pass in proto tcp from any to any port 8000:8005 keep state

(Опционально) Разрешаем SSH, если вы управляете Mac Studio удаленно в л
pass in proto tcp from any to any port 22 keep state

Use code with caution.

Шаг 2. Подключение правила в основной конфигурационный файл

Откройте главный файл настроек PF:

bash

sudo nano /etc/pf.conf

Use code with caution.

Найдите секцию с объявлением якорей (anchors) и добавьте строчку вниз:

text

anchor "fastapi.security"
load anchor "fastapi.security" from "/etc/pf.anchors/fastapi.security"

Use code with caution.

Важно: Строка `anchor` должна идти в блоке с другими анкерами, а `load anchor` — в самом конце файла.

Шаг 3. Запуск и применение правил файрвола

Проверьте файл на наличие синтаксических ошибок:

bash

sudo pfctl -vnf /etc/pf.conf

Use code with caution.

Если ошибок нет, принудительно включите файрвол и примените правила:

bash

sudo pfctl -e -f /etc/pf.conf

Use code with caution.

Как управлять этим файрволом:

- Отключить защиту (например, после завершения работы сервисов): `sudo pfctl -d`
- Включить защиту снова: `sudo pfctl -e -f /etc/pf.conf`
- Посмотреть статус и активные правила: `sudo pfctl -s info` или `sudo pfctl -s rules`

Это гарантирует, что пока ваши Docker-контейнеры активны и смотрят в глобальный интернет через проброшенный No-IP порт, сама операционная система macOS хоста будет надежно запечатана от любых внешних сканирований.

Если в процессе сборки `price_spider` возникнут проблемы с установкой зависимостей под архитектуру M4, дайте знать — мы добавим в Dockerfile необходимые компиляторы (`build-essential`). С какими еще сложностями вы сталкиваетесь при миграции?



version: '3.8' в yaml указывает версию чего? Это критично?
Выскочила ошибка:docker compose up -d --...

22:54

Ответ на вопрос про `version`

Строка `version: '3.8'` в файле `docker-compose.yml` указывает **версию схемы спецификации Docker Compose**. Она определяла, какие именно функции, ключевые слова и синтаксис поддерживаются парсером.

Слова и синтаксис поддерживаются парсером.

Критично ли это? Нет, не критично. [1, 4]

В современных версиях Docker (начиная со спецификации Compose Specification) этот атрибут официально **признан устаревшим (obsolete)** и полностью игнорируется. Docker Compose теперь автоматически использует самую актуальную версию схемы.

Предупреждение `the attribute 'version' is obsolete...` сообщает о том, что эту строку можно просто удалить из файла `docker-compose.yml`, чтобы оно больше не появлялось. На работу контейнеров это никак не повлияет.

Разбор и решение главной ошибки

Критическая ошибка заключается в следующей строке:

```
Cannot connect to the Docker daemon... Is the docker daemon running?
```

Это означает, что утилита `docker` в терминале не может найти запущенный движок

Docker (демон) по указанному пути

```
unix:///Users/v.ant/.docker/run/docker.sock
```

Поскольку вы написали, что **Docker Desktop запустили**, проблема на macOS чаще всего кроется в одной из трех причин. Выполните следующие шаги для исправления:

Шаг 1. Проверьте права на сокет (Самая частая проблема на macOS)

Docker Desktop на Mac создает символическую ссылку на сокет, но иногда у терминала или текущего пользователя нет прав на чтение/запись этого файла.

Выполните команду в терминале:

```
bash
```

```
sudo ln -sf $HOME/.docker/run/docker.sock /var/run/docker.sock
```

Use code with caution.



Затем перезапустите *Docker Desktop*.

Шаг 2. Включите Advanced-настройки в Docker Desktop

В Docker Desktop для Mac есть встроенный механизм управления сокетом, который иногда выключен по умолчанию.

1. Откройте **Docker Desktop**.
2. Нажмите на иконку **шестеренки (Settings)** в верхнем правом углу.
3. Перейдите во вкладку **Advanced**.
4. Убедитесь, что включена настройка **"Allow the default Docker socket to be used"** (или выберите режим *System* вместо *User*, если доступен выбор путей сокета).
5. Нажмите **Apply & restart**.

Шаг 3. Проверьте, что Docker Desktop работает корректно

Шаг 3. Проверьте переменные окружения терминала

Иногда терминал ищет Docker не там, где его запустил Docker Desktop. Проверьте, активен ли контекст, выполнив:

bash

docker context ls

Use code with caution.

В списке должна быть строка `default` или `desktop-linux` со звездочкой `*`. Если звездочка стоит на другом контексте, переключите его:

bash

docker context use default

Use code with caution.

Шаг 4. Финальная проверка

После выполнения шагов выполните простую команду для проверки связи с демоном:

bash

docker ps

Use code with caution.

Если команда вывела пустую таблицу (заголовки `CONTAINER ID`, `IMAGE` и т.д.) вместо ошибки — связь установлена. Теперь можно снова запускать сборку вашего FastAPI проекта:

bash

docker compose up -d --build

Use code with caution.

Удалось ли Docker Desktop пересоздать сокет после изменения настроек, и пошла ли сборка образов `shop_club` и `price_spider`?



docker compose up -d --build[+] Building 0.3s (4/4) FINISHED
=> [internal] load local bake definitions 0.0s => => reading ...

22:59

Ошибка сборки вызвана опечаткой на первой строке в файле

`/price_spider/Dockerfile`. По какой-то причине вместо полного адреса базового образа туда записалось `>>> FROM ://microsoft.com` (символы `>>>` и пропущенный протокол/название образа).

Docker пытается распарсить эту строку и завершает процесс аварийно.

Как исправить:

1. Откройте файл `/price_spider/Dockerfile` в VS Code или любом текстовом редакторе.
2. Полностью удалите первую строку и замените её на корректную:

dockerfile

```
FROM ://microsoft.com
```

Use code with caution.

Весь `Dockerfile` для сервиса `price_spider` в итоге должен выглядеть следующим образом:

dockerfile

```
FROM ://microsoft.com

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Установка системных зависимостей только для Chromium
RUN playwright install chromium

COPY . .
RUN rm -rf venv html_dumps screenshots logs/*

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8001"]
```

Use code with caution.

3. Сохраните файл и запустите сборку заново:

bash

```
docker compose up -d --build
```

Use code with caution.

После исправления этой строки Docker начнет скачивать базовые образы

после исправления этой строки Docker начнет скачивать базовые образы (`python:3.11-slim-bookworm` для первого сервиса и `playwright/python` для второго). Первая сборка образа со скачиванием Chromium внутри контейнера может занять пару минут.

Сообщите, если на этапе установки зависимостей (`pip install`) возникнут новые предупреждения. На каком этапе сейчас сборка?



Первая строка: `FROM ://microsoft.com` именно такая и была...
пожалуйста поясни что она делает?

23:09

Строка `FROM ://microsoft.com` в вашем файле — это результат **ошибки форматирования** или **опечатки**. На самом деле она **ничего полезного не делает и работать не может**, из-за чего Docker и выдал ошибку сборки.

Вот подробное объяснение, почему так произошло:

1. Что должна делать инструкция `FROM`?

Инструкция `FROM` в Dockerfile всегда указывает **базовый образ** (операционную систему и предустановленный язык программирования), на основе которого будет строиться ваш контейнер.

- **Как это должно выглядеть:** `FROM registry/repository/image:tag`
- **Пример рабочего варианта Microsoft для Playwright:** `FROM mcr.microsoft.com/playwright/python`

2. Почему синтаксис `FROM ://microsoft.com` сломал сборку?

1. **Отсутствие префикса:** Символы `://` используются для интернет-ссылок (URL), но Docker **не умеет** скачивать образы напрямую по URL-ссылкам веб-сайтов. Он работает со специальными хранилищами — Docker-реестрами (Docker Registry).
2. **Замена адреса:** В этой строке случайно стерлась самая важная часть адреса — `mcr.microsoft.com` (Microsoft Container Registry) и название самого образа (`playwright/python`). Остался только «огрызок» от домена.
3. **Реакция Docker:** Docker попытался прочитать двоеточие со слэшами как имя образа, не понял такой формат и выдал ошибку: `invalid reference format` (неверный формат ссылки).

Что нужно сделать прямо сейчас?

Эту строку нужно просто исправить, вернув туда полный и корректный адрес официального контейнера Microsoft, подготовленного специально для Python и веб-автоматизации (Playwright).

Откройте `/price_spider/Dockerfile` и замените эту дефектную строчку на:

```
dockerfile
```

```
FROM ://microsoft.com
```

Use code with caution.



После этого Docker поймет, что ему нужно зайти в официальный репозиторий Microsoft, скачать готовую сборку Linux Ubuntu (Jammy) с уже настроенным Python, и сборка вашего `price_spider` пойдет дальше без ошибок.

Удалось исправить строку? На каком этапе сборки контейнеров вы сейчас находитесь?



Microsoft Artifact Registry

Почему то строка всё время одинаковая и обрезанная... из слов я понял что она должна быть такойFROM ...



23:27

Нет, к сожалению, этот вариант тоже **не работает** 😞.

Дело в том, что Docker-клиент **не принимает протоколы** `http://` или `https://` в инструкции `FROM`. Из-за этого сборка снова завершится ошибкой синтаксиса. Ссылка, которую вы нашли (`https://mcr.microsoft.com/...`), — это веб-адрес каталога Microsoft для просмотра через обычный браузер.

Для самого Docker адрес пишется в специальном формате: **без протокола** `https://` и обязательно **с указанием версии (тега)** через двоеточие в конце.

Как правильно написать эту строку:

Замените первую строчку в вашем `/price_spider/Dockerfile` ровно на этот текст:

```
dockerfile
```

```
FROM ://microsoft.com
```

Use code with caution.



Разбор строки по частям:

- `mcr.microsoft.com` — это адрес самого сервера-хранилища (Microsoft Container Registry), но без префикса `https://`.
- `/playwright/python` — имя нужного репозитория с готовым окружением.
- `:v1.42.0-jammy` — это **тег (tag)**, то есть конкретная версия. Она жестко фиксирует версию Playwright (`v1.42.0`) и версию операционной системы Linux (`jammy` — это кодовое имя Ubuntu 22.04 LTS). Без указания тега Docker либо скачает непредсказуемую версию, либо вообще выдаст ошибку.

Попробуйте сохранить именно этот вариант, после чего запустите сборку в терминале:

```
bash
```

```
docker compose up -d --build
```

Use code with caution.



Если код скопирован корректно, Docker успешно начнет загрузку (Pull) базовых слоев от Microsoft. Напишите, сдвинулся ли процесс сборки дальше?



Вроде запустились, но для начала не получается обратиться по localhost:8000docker ps CONTAINER ID ...

23:44

Контейнеры запустились, но сразу после старта упали с ошибкой и теперь бесконечно перезапускаются. Об этом говорит статус **Restarting (1) 38 seconds**

ago и отсутствие портов в колонке **PORTS**.

Пока контейнер находится в состоянии циклической перезагрузки, обратиться к нему по **localhost:8000** не получится.

Основная причина падения FastAPI при первом переносе в Docker — **несовпадение путей импорта** (из-за чего Python не может найти файл **main.py**) или **отсутствие обязательных переменных окружения**.

Давайте быстро выясним точную причину падения.

Шаг 1. Посмотрите логи контейнеров

Выполните в терминале команду, чтобы увидеть конкретную ошибку, которую выдает Python при старте:

```
bash
```

```
docker logs shop_club_first
```

Use code with caution.



(Если там пусто или ничего непонятно, проверьте второй: `docker logs price_spider_second`).

Шаг 2. Самые частые причины и как их исправить

Судя по структуре вашего проекта из VS Code, скорее всего, дело в одной из двух проблем:

Проблема А: Пути запуска Uvicorn

В структуре проекта файл **main.py** лежит внутри папки **app/**. В файле **Dockerfile** мы указали рабочую директорию **WORKDIR /app** и команду **uvicorn app.main:app**. Если Docker скопировал содержимое папки **shop_club** целиком, то внутри контейнера путь к файлу стал **/app/app/main.py**. Из-за этого Uvicorn падает с

контейнера путь к файлу стал `/app/app/main.py`. из-за этого uvicorn падает с ошибкой `ModuleNotFoundError`.

Как исправить:

Попробуйте изменить последнюю строку (`CMD`) в **обоих** `Dockerfile`:

- В `/shop_club/Dockerfile`:

dockerfile

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Use code with caution.

(Если не работает, верните обратно и попробуйте: `CMD ["python", "-m", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]`)

- В `/price_spider/Dockerfile`:

dockerfile

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]
```

Use code with caution.

Проблема Б: Отсутствие переменных в `.env`

Если в вашем коде (например, в файле `core/config.py`) используется библиотека `pydantic` для конфигурации, и какая-то важная переменная (секретный ключ, настройки БД) отмечена как обязательная, но отсутствует в созданном файле `.env` в корне проекта, FastAPI аварийно завершит работу до старта сервера.

Шаг 3. Перезапуск

После корректировки `Dockerfile` или `.env` примените изменения:

bash

```
docker compose down
docker compose up -d --build
```

Use code with caution.

Пришлите, пожалуйста, **текст ошибки из команды** `docker logs shop_club_first` — он сразу укажет на конкретную строку кода или модуля, из-за которой падает приложение. На что ругается Python в логах?



Очень длинный log но в конце `ModuleNotFoundError: No module named 'psycopg2'`

23:58

Отлично, это классическая и очень легко решаемая проблема при переносе Python-приложений в Docker.

Почему это произошло?

Модуль `psycopg2` — это драйвер для работы с базой данных PostgreSQL. Когда проект работал локально на вашем Mac Studio, этот модуль был установлен в вашем виртуальном окружении (`venv`). Однако внутри Docker-контейнера используется чистая операционная система Linux (Debian/Ubuntu), где этого модуля по умолчанию нет.

Так как мы удалили папку `venv` ради оптимизации веса контейнера, Docker пытается

установить все библиотеки из файла `requirements.txt`. Ошибка `ModuleNotFoundError: No module named 'psycopg2'` означает, что эта библиотека либо **забыта** в `requirements.txt`, либо упала с ошибкой во время установки (так как для сборки `psycopg2` из исходников в Linux требуются системные компиляторы).

Как исправить ситуацию правильно и быстро

Для Docker и архитектуры ARM64 (Mac Studio M4) лучшим решением будет использование бинарной (уже скомпилированной) версии этого драйвера — `psycopg2-binary`. Она ставится мгновенно и не требует установки тяжелых системных утилит внутрь контейнера.

Шаг 1. Обновите файлы зависимостей

Вам нужно добавить этот модуль в списки зависимостей **обоих** ваших сервисов, так как они оба работают с базой данных.

1. Откройте файл `/shop_club/requirements.txt` и добавьте в самый конец новую строку:

text

```
psycopg2-binary>=2.9.0
```

Use code with caution.



2. Откройте файл `/price_spider/requirements.txt` и сделайте то же самое — добавьте в конец:

text

```
psycopg2-binary>=2.9.0
```

Use code with caution.



(Примечание: Если в файлах уже была строка `psycopg2`, просто замените её на `psycopg2-binary`).

Шаг 2. Пересоберите проект

Теперь заставьте Docker заново прочитать измененные файлы `requirements.txt` и

пересобрать слои контейнеров. Запустите команду в терминале:

```
bash
```

```
docker compose up -d --build
```

Use code with caution.



Шаг 3. Проверка статуса

Подождите секунд 10-15, пока завершится сборка, и проверьте статус контейнеров:

```
bash
```

```
docker ps
```

Use code with caution.



Если в колонке `STATUS` вместо циклического `Restarting` появилось заветное `Up ... seconds (or minutes)`, а в колонке `PORTS` отобразилось `0.0.0.0:8000->8000/tcp`, значит, проблема полностью решена!

Попробуйте выполнить пересборку. Изменился ли статус контейнеров на `Up`? Если нет, то какая новая ошибка появилась в `docker logs shop_club_first`?



Ну да проект асинхронно

работаетsqlalchemy.exc.InvalidRequestError: The asyncio ...

00:06

Абсолютно верно! Раз ваш FastAPI-проект асинхронный, он использует `AsyncSession` из SQLAlchemy. Стандартный драйвер `psycopg2` предназначен только для синхронного кода, и SQLAlchemy справедливо блокирует запуск.

Для асинхронной работы с PostgreSQL в Python-среде и Docker стандартом является драйвер `asyncpg`.

Давайте перенастроим конфигурацию на асинхронные рельсы.

Шаг 1. Меняем драйвер в зависимостях

Вам нужно заменить синхронный драйвер на асинхронный в файлах зависимостей **обоих** сервисов.

1. Откройте `/shop_club/requirements.txt`. Если там есть `psycopg2` или `psycopg2-binary`, **удалите их**. Вместо них добавьте:

```
text
```

```
asyncpg>=0.29.0
```

Use code with caution.



2. Откройте `/price_spider/requirements.txt` и сделайте точно так же — оставьте только:

```
text
```

```
asyncpg>=0.29.0
```

Use code with caution.



Шаг 2. Корректируем строку подключения в `.env`

Поскольку Docker теперь будет использовать драйвер `asyncpg`, нам нужно явно указать это в протоколе строки подключения к базе данных (URL). В асинхронной SQLAlchemy протокол должен начинаться с `postgresql+asyncpg://`.

Откройте ваш файл `.env` в корне проекта и обновите переменную `DATABASE_URL`:

```
env
```

```
# Было: postgresql://user:password@host.docker.internal:5432/your_db_name
# Стало:
DATABASE_URL=postgresql+asyncpg://user:password@host.docker.internal:5432,
```

Use code with caution.



(Замените `user`, `password` и `your_db_name` на ваши реальные доступы к PostgreSQL на macOS).

Шаг 3. Пересборка и запуск

Теперь очистим старые слои контейнеров и соберем их с новым асинхронным драйвером:

```
bash
```

```
docker compose down
docker compose up -d --build
```

Use code with caution.



Шаг 4. Проверка

Подождите 10 секунд и проверьте статус:

```
bash
```

```
docker ps
```

Use code with caution.



Если статус сменился на `Up`, попробуйте открыть в браузере страницу документации вашего внешнего сервиса: `http://localhost:8000/docs`.

Завелся ли асинхронный движок? Каковы результаты в `docker ps`?



Снова ругался что нет psycopg и я для теста добавил не удаляя asynpcpg>=0.29.0 Говорит не asynpc, то есть два ...



00:44

Эта ошибка возникает из-за того, как именно ваша программа (а точнее, SQLAlchemy) пытается подключиться к базе данных. Наличие обоих драйверов в `requirements.txt` разрешено, но **одновременно использовать их в одной строке подключения нельзя**.

Вот почему SQLAlchemy ругается на `psycopg2`, даже когда `asynpcpg` установлен:

В чем причина?

SQLAlchemy определяет, какой драйвер использовать, исключительно по **тексту строки подключения (URL базы данных)**, которую вы ей передаете из файла `.env` или `config.py`.

- Если строка начинается с `postgresql://...`, SQLAlchemy по умолчанию подтягивает синхронный драйвер `psycopg2`.
- Когда вы запускаете асинхронный метод (`asyncio`), SQLAlchemy видит, что загружен синхронный `psycopg2`, и падает с ошибкой: *"The loaded 'psycopg2' is not async"*.

Даже если в системе установлен `asynpcpg`, SQLAlchemy **не переключится на него сама**, пока вы явно не измените протокол в строке подключения.

Как это исправить за 2 шага

Шаг 1. Исправьте файл `.env` в корне проекта

Вам нужно принудительно заставить SQLAlchemy использовать асинхронный драйвер. Для этого измените строку подключения в вашем файле `.env`:

```
env
```

```
# Было (или похожее):
DATABASE_URL=postgresql://v.ant:password@host.docker.internal:5432/shop_c

# ДОЛЖНО БЫТЬ (добавляем +asyncpg):
DATABASE_URL=postgresql+asyncpg://v.ant:password@host.docker.internal:5432
```

Use code with caution.



(Обязательно проверьте, чтобы префикс был именно `postgresql+asyncpg://`).

Шаг 2. Проверьте файлы конфигурации (Если шаг 1 не помог)

Иногда в коде (например, в файле `app/core/config.py` или `app/database.py`) строка подключения жестко зашита вручную (hardcoded) или модифицируется кодом.

Убедитесь, что SQLAlchemy инициализирует движок `create_async_engine` именно с асинхронным `postgresql+asyncpg`. Пример правильной инициализации в коде:

```
python
```

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession

# Переменная должна содержать строку с +asyncpg
engine = create_async_engine(settings.DATABASE_URL, echo=True)
```

Use code with caution.



Почему проект запрашивал `psycopg2` изначально?

Вы упомянули: "Снова ругался что нет `psycopg2`".

В вашей структуре проекта присутствует папка `alembic/` (инструмент для миграций базы данных). По умолчанию Alembic генерирует миграции в **синхронном** режиме и требует именно `psycopg2`.

Поэтому идеальный и рабочий состав `requirements.txt` для вашего проекта — это **наличие обоих драйверов**, но разделение их обязанностей:

1. `psycopg2-binary` останется в системе, чтобы работал **Alembic** (для применения миграций).
2. `asyncpg` будет использоваться самим **FastAPI** приложением для асинхронных запросов.

Что делать сейчас:

1. Оставьте оба драйвера в `requirements.txt`.
2. Измените `DATABASE_URL` в вашем `.env` файле, добавив туда `+asyncpg`.
3. Перезапустите контейнеры:

```
bash
```

```
docker compose down
```

```
docker compose up -d --build
```

Use code with caution.



После этого SQLAlchemy переключит FastAPI на асинхронный режим работы. Проверьте статус контейнеров через `docker ps`. Изменился ли статус на `Up`?



Везде поправил, а в корне проекта `.env` действительно остался синхронным!но ошибки ...



00:51

Отлично, это означает, что **асинхронное подключение успешно настроилось** и FastAPI достучался до вашего сервера PostgreSQL! Теперь мы упёрлись в настройки безопасности самой СУБД.

Ошибка `InvalidAuthorizationSpecificationError: role "user" does not exist` говорит о том, что в строке подключения указан пользователь `user`, но в базе данных PostgreSQL на вашем Mac Studio **нет пользователя с таким именем**.

Почему это произошло?

Скорее всего, в вашем файле `.env` в корне проекта в строке `DATABASE_URL` в качестве примера остался шаблонный пользователь `user` (например, `postgresql+asyncpg://user:password@...`).

Когда вы запускали проект локально в VS Code, код мог брать имя пользователя вашей macOS (судя по путям, это `v.ant`) или стандартного системного пользователя `postgres`.

Как исправить:

Шаг 1. Выясните реальное имя пользователя вашей БД

Вспомните, под какими учётными данными вы работаете с базой в VS Code или через графический клиент (например, pgAdmin / DBeaver). Чаще всего это:

- `postgres` (суперпользователь PostgreSQL по умолчанию).
- `v.ant` (имя вашей учетной записи в macOS, если Postgres ставился через Homebrew).

Шаг 2. Обновите `.env` в корне проекта

Откройте `.env` и замените шаблонное слово `user` на ваше настоящее имя пользователя и укажите его актуальный пароль.

Пример, если ваш пользователь — `postgres`:

env

```
DATABASE_URL=postgresql+asyncpg://postgres:ваш_пароль@host.docker.internal:
```

Use code with caution.



Пример, если ваш пользователь совпадает с именем пользователя Mac:

env

```
DATABASE_URL=postgresql+asyncpg://v.ant:ваш_пароль@host.docker.internal:5:
```

Use code with caution.



Шаг 3. Перезапустите контейнеры

При изменении файла `.env` контейнеры нужно перезагрузить, чтобы они подтянули новые переменные окружения:

bash

```
docker compose down  
docker compose up -d
```

Use code with caution.



(Параметр `--build` здесь уже не нужен, так как сам код и зависимости не менялись).

Проверка результата

После перезапуска проверьте статус командой:

bash

```
docker ps
```

Use code with caution.



Если статус изменился на **Up**, поздравляю — миграция в Docker успешно завершена! Вы можете открыть `http://localhost:8000/docs` и проверить работу веб-интерфейса.

Применились ли новые параметры авторизации? На каком этапе сейчас запуск приложения?



Да действительно в .env снова недоделал... исправил
Теперь
docker compose up -d --build?

00:56

Раз вы уже исправили имя пользователя и пароль в файле `.env`, использовать флаг `--build` **необязательно**, так как сам код приложений и их зависимости (`requirements.txt`) не менялись.

Однако, чтобы Docker Compose гарантированно сбросил старые параметры окружения и применил новые данные авторизации, лучше всего выполнить перезапуск через полную остановку:

bash

```
docker compose down
docker compose up -d
```

Use code with caution.



Что делать сразу после запуска:

1. Проверьте статус контейнеров:

bash

```
docker ps
```

Use code with caution.



Убедитесь, что оба контейнера (`shop_club_first` и `price_spider_second`) имеют статус **Up (сколько-то секунд/минут)** и не уходят в перезагрузку.

2. Проверьте логи (если контейнеры снова упадут):

bash

```
docker logs shop_club_first
```

Use code with caution.



Если база данных авторизовала пользователя, вы увидите логи успешного запуска Uvicorn: `INFO: Uvicorn running on http://0.0.0.0`.

3. Проверьте доступность из браузера:

Перейдите по адресу `http://localhost:8000/docs` или по вашему домену `http://ddns.net`, чтобы протестировать интерактивную документацию FastAPI.

Контейнеры стабилизировались в статусе **Up**? Удалось открыть главную страницу или документацию?



Здорово! Старт есть, приложение отвечает на запросы из интернета, но есть серьезная проблема. В VS Code Chro... ▾

01:17

Поздравляю с успешным запуском контейнеров в сети! 🚀 То, что приложение уже отвечает из интернета — это огромный шаг. Появление антибот-защиты (Cloudflare, Variti и др.) при переносе в Docker — это стандартная ситуация. Когда Playwright запускается внутри Docker, он теряет историю, куки, специфичные цифровые отпечатки (fingerprints) вашей macOS и выглядит для сайтов как «голый» подозрительный Linux-скрипт.

Ниже разбор, как смотреть логи, как правильно почистить и пробросить ваш профиль, а также как победить антибот в Docker на архитектуре M4.

1. Как смотреть логи приложения в реальном времени

Чтобы не просто читать статический лог, а видеть запросы и ошибки прямо в момент их возникновения (как в консоли VS Code), используйте флаг `-f` (follow):

```
bash
```

```
# Логи внешнего сайта
```

```
docker logs -f shop_club_first
```

```
# Логи поискового спайдера (здесь будут ошибки Playwright и антибота)
```

```
docker logs -f price_spider_second
```

Use code with caution.



Чтобы выйти из режима просмотра логов, нажмите `Ctrl + C`.

2. Как очистить профиль Chrome перед переносом (с 1.5 ГБ до ~50 МБ)

Переносить 1.5 ГБ внутрь контейнера нельзя — это сломает всю оптимизацию Docker. Нам нужно удалить весь кэш картинок, стилей, медиа и автозаполнения, но **сохранить сессии (куки) и авторизацию на маркетплейсах**.

Папка профиля Chrome (обычно `Default` или `chrome_profile`) содержит много мусора. Удалите из нее следующие папки (через Finder или Терминал):

- `Cache/` и `Code Cache/` (это 80% веса — кэш картинок и скриптов сайтов)
- `GPUCache/` (кэш шейдеров видеокарты, в Docker он вообще не нужен)
- `Service Worker/` (кэш фоновых обновлений страниц)

После такой очистки папка вашего профиля со всеми куками и авторизациями сожмется до **30–50 Мегабайт**.

3. Как подложить профиль в Docker-контейнер

В Docker файлы профиля **нельзя** запекать внутрь образа (через `COPY`), так как браузер постоянно пишет туда новые данные. Профиль нужно **примонтировать** как общую папку между вашим Mac и контейнером с помощью `volumes` в Docker Compose.

1. Перенесите очищенную папку `chrome_profile` в корень вашего проекта (туда, где лежит `docker-compose.yml`).
2. Откройте файл `docker-compose.yml` и добавьте секцию `volumes` для сервиса `price_spider`:

yaml

```
price_spider:
  build:
    context: ./price_spider
  container_name: price_spider_second
  restart: always
  env_file:
    - .env
  extra_hosts:
    - "host.docker.internal:host-gateway"
  # МОНТИРУЕМ ПРОФИЛЬ:
  volumes:
    - ./chrome_profile:/app/chrome_profile_docker
```

Use code with caution.

3. В коде вашего спайдера (файл `price_spider/app/services/playwright_base.py` или `spider.py`), там где вы инициализируете контекст браузера, укажите этот новый путь к профилю:

python

```
# Было локально: что-то вроде playwright.chromium.launch_persistent_context
# Стало для Docker:
context = await playwright.chromium.launch_persistent_context(
    user_data_dir='/app/chrome_profile_docker',
    headless=True, # или False, если используете виртуальный дисплей (см.
    # ... ваши остальные параметры
)
```

Use code with caution.

4. Почему один лишь профиль может не спасти от антибота в Docker (и как решить)

В отличие от macOS, внутри Docker-образа от Microsoft по умолчанию передаются явные признаки автоматизации (переменная `navigator.webdriver = true` специфичные системные шрифты Linux, отсутствие графического экрана). Антиботы это видят.

Чтобы ваш `price_spider` в Docker работал так же успешно, как в VS Code на Mac

Studio, примените три золотых правила:

Решение А: Поставьте библиотеку `playwright-stealth`

Она маскирует Docker-контейнер под обычный домашний браузер, подменяя все системные переменные Linux на человеческие.

1. В `/price_spider/requirements.txt` добавьте:

text

```
playwright-stealth>=1.0.6
```

Use code with caution.



2. В коде инициализации браузера (`spider.py`) примените маскировку к созданной странице:

python

```
from playwright_stealth import stealth_async

page = await context.new_page()
await stealth_async(page) # Маскируем страницу под реального пользователя
await page.goto("https://ozon.ru...")
```

Use code with caution.



Решение Б: Передавайте User-Agent вашего Mac Studio

Если антибот видит в заголовках `Linux x86_64`, а отпечаток процессора говорит, что это Apple Silicon, происходит блокировка. Передайте в параметры запуска `launch_persistent_context` User-Agent вашего реального Chrome с Mac:

python

```
user_agent = "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit,

context = await playwright.chromium.launch_persistent_context(
    user_data_dir='/app/chrome_profile_docker',
    user_agent=user_agent,
    # ...
)
```

Use code with caution.



Решение В: Имитация реального экрана (Xvfb)

Некоторые жесткие антиботы блокируют запросы, если у браузера физически нет окна отрисовки (Headless-режим в Linux выдает себя размером экрана 0x0).

Если предыдущие шаги не помогут, в `Dockerfile` для `price_spider` можно будет

добавить утилиту `xvfb` (виртуальный монитор в памяти контейнера). Тогда Playwright будет думать, что он работает в полноценном графическом режиме `headless=False`, хотя физически окна на вашем Mac появляться не будет.

Что делать сейчас:

1. Очистите папку профиля на Mac.
2. Подключите её через `volumes` в `docker-compose.yml`.
3. Добавьте `playwright-stealth` в зависимости.
4. Выполните полную пересборку: `docker compose down && docker compose up -d --build`.

Запустите параллельно лог `docker logs -f price_spider_second` и посмотрите, какие именно ошибки начнет писать спайдер при попытке зайти на сайт. Какие сайты вас блокируют?



В моем приложении много отладочного кода, например формируются скриншоты при ошибках сети например и ...

01:28

Да, посмотреть эти скриншоты можно прямо сейчас, и для этого даже не обязательно заново пересобирать контейнер! Docker сохраняет всё, что генерирует приложение, внутри изолированной файловой системы запущенного контейнера.

Вот **три способа** вытащить и посмотреть ваши отладочные скриншоты прямо сейчас, а также инструкция, как сделать так, чтобы они автоматически падали в удобную папку на вашем Mac Studio.

Способ 1. Скопировать скриншоты из контейнера на Mac (Самый быстрый)

У Docker есть встроенная команда `cp`, которая умеет копировать файлы и папки изнутри контейнера наружу на хост-машину, даже если папка не была примонтирована.

Судя по структуре вашего проекта, скриншоты в спайдере создаются в папке `screenshots/`. Выполните в терминале Mac следующую команду:

```
bash
```

```
docker cp price_spider_second:/app/screenshots ./container_screenshots
```

Use code with caution.



Что произойдет: В текущей папке на вашем Mac появится директория `container_screenshots`, внутри которой будут лежать все картинки, которые успел сделать Playwright с момента запуска контейнера. Откройте их через Finder и посмотрите, какую именно заглушку антибота (капчу или Cloudflare) он встретил.

(Если папка пуста или вы не уверены, куда именно код сохраняет файлы, см. Способ 2).

Способ 2. Зайти внутрь контейнера через «терминал в терминале»

Вы можете подключиться к файловой системе контейнера в интерактивном режиме и посмотреть, где лежат файлы и сколько они весят:

```
bash
```

```
docker exec -it price_spider_second bash
```

Use code with caution.



Вы окажетесь внутри Linux-окружения вашего контейнера в папке `/app`. Теперь вы можете использовать стандартные команды:

- `ls -la` — посмотреть список файлов и папок.
- `ls -la screenshots/` — проверить наличие скриншотов.
- `exit` — выйти обратно в терминал macOS.

Способ 3. Посмотреть через графический интерфейс Docker Desktop

Поскольку у вас установлен Docker Desktop, это можно сделать буквально мышкой:

1. Откройте окно **Docker Desktop**.
2. Перейдите во вкладку **Containers** и нажмите на контейнер `price_spider_second`.
3. Сверху выберите вкладку **Files**.
4. Найдите папку `app` ➔ `screenshots`. Там будут видны все ваши `.png` файлы.
5. Нажмите правой кнопкой мыши на любой скриншот и выберите **Save as...**, чтобы сохранить его на Mac и открыть через Просмотр (Preview).

Как настроить, чтобы скриншоты, дампы и логи всегда сохранялись на Mac

Чтобы не вытаскивать файлы вручную каждый раз, когда «вас раскусили», давайте добавим их в секцию `volumes` вашего файла `docker-compose.yml`. Это свяжет папки внутри контейнера с папками на вашем Mac, и файлы будут появляться в Finder мгновенно в реальном времени.

Откройте `docker-compose.yml` и обновите конфигурацию для `price_spider`:

```
yaml
```

```
price_spider:
  build:
```

```

context: ./price_spider
container_name: price_spider_second
restart: always
env_file:
  - .env
extra_hosts:
  - "host.docker.internal:host-gateway"
volumes:
  - ./chrome_profile:/app/chrome_profile_docker
  # ДОБАВЛЯЕМ ПАПКИ ДЛЯ ОТЛАДКИ:
  - ./price_spider/screenshots:/app/screenshots
  - ./price_spider/html_dumps:/app/html_dumps
  - ./price_spider/logs:/app/logs

```

Use code with caution.

**Что сделать после изменения `docker-compose.yml`:**

1. Убедитесь, что локальные папки `screenshots`, `html_dumps` и `logs` физически существуют в вашей папке `price_spider/` на Mac. Если вы их удалили ранее, просто создайте пустые папки.
2. Примените конфигурацию:

bash

docker compose up -d

Use code with caution.



(Пересборка `--build` не нужна, мы просто обновили правила монтирования папок).

Теперь, как только Playwright сделает скриншот ошибки при парсинге Ozon или Wildberries, файл **тут же появится в папке проекта в VS Code**, и вы сможете сразу открыть его встроенным просмотрщиком картинок.

Какую именно картину вы увидели на первых скриншотах? Там стандартная капча, «Доступ ограничен (403)» или бесконечная загрузка Cloudflare?



Очистил и поместил `chrome_profile` в корень проекта (vs)
`/app/chrome_profile_docker` папка в контейнере становится ...

Q2:17

Она остаётся полностью рабочей! Более того, именно **через монтирование (volumes)** папка и должна работать в Docker.

Когда вы добавили строку `./chrome_profile:/app/chrome_profile_docker` в `docker-compose.yml`, Docker взял вашу физическую очищенную папку с Mac Studio и виртуально «подставил» её внутри контейнера по пути `/app/chrome_profile_docker`.

Однако, чтобы всё функционировало без сбоев на архитектуре M4, нужно учесть **два критически важных нюанса**, из-за которых профиль внутри контейнера

действительно может стать «нерабочим» или заблокированным.

Нюанс 1. Браузер на Mac должен быть полностью закрыт

Профили Chromium устроены так, что внутри них создаются файлы блокировки сессии (например, файлы `SingletonLock`, `SingletonCookie` или `LOCK`).

- **Проблема:** Если во время запуска контейнера на вашем Mac запущен обычный Google Chrome (или если вы не закрыли проект в VS Code, который держал локальный инстанс браузера), эти файлы заблокированы хост-системой.
- **Результат в Docker:** Контейнер попытается прочитать профиль, наткнется на `LOCK`-файл вашей macOS и Playwright выдаст ошибку: «*Target page, context or browser has been closed*» или намертво зависнет.

Что делать: Перед выполнением `docker compose up` полностью закройте Chrome на Mac (`Cmd + Q`).

Нюанс 2. Очистка системных файлов блокировки

В процессе очистки профиля от кэша (когда вы удаляли `Cache` и `GPUCache`), обязательно удалите файлы, которые Chromium создаёт для защиты от одновременного запуска.

Зайдите в вашу локальную папку `chrome_profile` на Mac (в корне проекта) и удалите, если они там есть:

- Файл `LOCK` (без расширения)
- Файл `SingletonLock`
- Файл `SingletonSocket`

После этого Docker сможет использовать профиль как «свой собственный».

Нюанс 3. Проверьте правильность пути в коде `price_spider`

Убедитесь, что в коде инициализации Playwright вы исправили путь на внутренний (контейнерный). Docker ничего не знает о путях вашего Mac, он видит только то, что внутри него.

В Python-коде запуска спайдера должно быть строго так:

python

```
context = await playwright.chromium.launch_persistent_context(
    user_data_dir='/app/chrome_profile_docker', # Путь из правой части ди
    headless=True,
    # Ваши аргументы против антибота
    args=[
        '--disable-blink-features=AutomationControlled',
        '--no-sandbox'
    ]
)
```

Use code with caution.



Как проверить, подцепился ли профиль?

Запустите контейнеры и посмотрите лог:

```
bash
docker logs price_spider_second
```

Use code with caution.

Если в логах нет ошибок инициализации браузера (Target closed), значит, папка /app/chrome_profile_docker успешно примонтировалась и работает.

Теперь, когда профиль на месте, антибот перестал блокировать запросы, или на скриншотах всё ещё видна защита?

